

Day 51 -- Line-by-Line Syntax Explanation

day51_l1_l2_regularization_cnn.py, segment by segment, line by line. Pure Python/NumPy syntax focus, with the math each chunk implements, a system diagram of how it works, and real (rerun, not fabricated) graphs showing the code+math turned into a visualization.

MD Mutasim Billah | 52-week Data Science → ML/AI roadmap | Week 8, Day 3 reference

This document walks the script top to bottom in the order it actually runs. Each segment matches one numbered section of the script.

Segment 0 -- Imports and Shared Helpers (reused verbatim from Days 39-50)

```
import numpy as np
import matplotlib
matplotlib.use("Agg")
import matplotlib.pyplot as plt
np.random.seed(42)
def relu(z): return np.maximum(0, z)
def drelu(z): return (z > 0).astype(z.dtype)
def softmax(z): ...
def one_hot(y, n_classes): ...
def cross_entropy_loss(probs, y_onehot): ...
def he_scale(n_in): return np.sqrt(2.0 / n_in)
def conv2d_forward_mc(X, W, b): ...
def conv2d_backward_mc(dZ, X, W): ...
def maxpool_forward(A, size=2, stride=2): ...
def maxpool_backward(dOut, A, idx_cache, size=2, stride=2): ...
def iterate_minibatches(X, y_onehot, batch_size, rng, shuffle=True): ...
def make_junction_dataset_rgb(...): ...
def dropout_forward(A, keep_prob, training, rng): ...
def dropout_backward(dOut, mask): ...
def train_with_history(net, X, y, X_test, y_test, epochs, ...): ...
```

Every function here is unchanged from Days 49-50, dropout_forward/backward included -- dropout is held at keep_prob=1.0 all day today (an explicit forward() argument, not a code change), so these functions are present only because AlexNetStyleConvNet's forward() calls them unconditionally.

Segment 1 -- AlexNetStyleConvNet, Now With L1/L2 (NEW today)

```
def __init__(self, ..., lam=0.0, reg_type=None, seed=42):
    ...
    self.lam, self.reg_type = lam, reg_type
```

Two new constructor arguments -- `lam` (a float, the regularization strength) and `reg\_type` (a string 'l2'/'l1', or None for no regularization) -- stored directly on `self` so backward() can read them without changing backward()'s own call signature.

```
weight_grads = [(self.Wc1, dWc1), (self.Wc2, dWc2), (self.Wc3, dWc3),
                 (self.W1, dW1), (self.W2, dW2), (self.W3, dW3)]
if self.reg_type == "l2":
    for W, dW in weight_grads:
        dW += self.lam * W
elif self.reg_type == "l1":
    for W, dW in weight_grads:
        dW += self.lam * np.sign(W)
```

This block sits AFTER every dW/db is computed by the usual backprop chain, and BEFORE the final `p -= self.lr \* g` update loop -- so the regularization penalty is added to the gradient exactly once, in one place, regardless of which conv or FC layer it belongs to. `weight\_grads` deliberately excludes every bias tensor (bc1, bc2, bc3, b1, b2, b3) -- only WEIGHTS get regularized, matching Day 38's own convention.

```
def loss_with_penalty(self, X, y_onehot):
    probs = self.forward(X, training=False)
    ce = cross_entropy_loss(probs, y_onehot)
```

```

if self.reg_type == "l2":
    penalty = sum(0.5 * self.lam * np.sum(W ** 2) for W in [...])
elif self.reg_type == "l1":
    penalty = sum(self.lam * np.sum(np.abs(W)) for W in [...])
else:
    penalty = 0.0
return ce + penalty

```

A NEW method, used only by the gradient check -- it exists because the numeric side of a gradient check needs to perturb the SAME total loss the analytic gradient describes, and that total loss (cross-entropy + penalty) is different from plain `cross_entropy_loss` whenever `reg_type` is set.

SYSTEM DIAGRAM -- backward() with the new regularization block: ordinary backprop computes every `dW`, the L1/L2 block adds a penalty term to the WEIGHT gradients only, then the usual SGD update runs.

Segment 2 -- Gradient Check, Two Regularization Types

```

for reg_type, lam in [("l2", 0.05), ("l1", 0.05)]:

```

```

    ...
    if reg_type == "l2":
        dw1 = dw1 + lam * net.W1
    else:
        dw1 = dw1 + lam * np.sign(net.W1)
    ...
    lp = net.loss_with_penalty(X, y_oh)
    lm = net.loss_with_penalty(X, y_oh)

```

The check runs TWICE in one loop, once per `reg_type`, building the analytic gradient by hand to match whichever penalty is active, then comparing against a numeric estimate computed from `loss_with_penalty` (not plain cross-entropy) at each perturbed point -- the same full-pipeline convention every gradient check since Day 44 has used, extended to cover the new penalty term too.

Segment 3 -- The Lambda Sweep

```

configs = [
    ("none (lam=0)", None, 0.0),
    ("L2 lam=5e-5", "l2", 5e-5),
    ("L2 lam=1e-4", "l2", 1e-4),
    ("L2 lam=1e-2", "l2", 1e-2),
    ("L1 lam=1e-3", "l1", 1e-3),
    ("L1 lam=5e-3", "l1", 5e-3),
    ("L1 lam=1e-2", "l1", 1e-2),
]
results = {}
for name, reg_type, lam in configs:
    net = AlexNetStyleConvNet(..., lam=lam, reg_type=reg_type, seed=42)
    hist = train_with_history(net, X_train, y_train, X_test, y_test, ...)
    results[name] = dict(net=net, hist=hist,
                        train_acc=hist["train_acc"][-1],
                        test_acc=hist["test_acc"][-1])

```

A list of (name, reg_type, lambda) tuples drives one uniform loop -- adding another lambda value to sweep means adding one more tuple to `configs`, not writing new training code. Every config's full `net` object AND its `hist` (per-epoch trajectory) are kept in `results`, not just the final numbers -- Segment 4's sparsity check needs the trained `net`, and the plot needs each config's own `hist`.

```

def sparsity(net, threshold=0.01):
    weights = np.concatenate([net.Wc1.ravel(), net.Wc2.ravel(),
                              net.Wc3.ravel(), net.W1.ravel(),
                              net.W2.ravel(), net.W3.ravel()])
    return float(np.mean(np.abs(weights) < threshold))

```

NEW today: flattens and concatenates every weight tensor (`.ravel()` on each, then `np.concatenate()`) into one long 1-D array, so a single `np.mean(np.abs(weights) < threshold)` measures sparsity across the WHOLE network at once

rather than layer by layer.

SYSTEM DIAGRAM -- one sweep config's pipeline: construct with (lam, reg_type) -> train_with_history -> read final train/test_acc and measure weight sparsity.

Segment 4 -- What This Maps To in PyTorch, and main()

```
def main():  
    demo_gradient_check()  
    demo_lambda_sweep()  
    note_on_pytorch()
```

Same ordering principle every earlier day has followed: verify the new code first, then run the real experiment, then relate it back to a production framework.

Quick Syntax Glossary -- New or Reinforced Constructs

``self.lam, self.reg_type = lam, reg_type`` (config stored on self, read later by `backward()`)

New today: two plain constructor arguments stashed on the instance so a LATER method (`backward()`) can read them without them being passed through every intermediate call -- the same cache-on-self pattern already used for `self.keep_prob` (Day 49) and `self.lr`, extended to regularization config.

``if self.reg_type == "l2": ... elif self.reg_type == "l1": ...`` (one code path selects between two math formulas)

A single `backward()` implementation serves three behaviors (no regularization, L2, L1) by branching on a string flag rather than needing three separate `backward()` methods or three separate classes -- the same 'training=' flag philosophy Day 49 used for dropout, generalized to a multi-way choice.

``np.concatenate([t.ravel() for t in [...]])`` (flatten and merge several arrays of different shapes into one)

``t.ravel()`` flattens each weight tensor (different shapes: `Wc1` is 4-D, `W1` is 2-D) down to 1-D first, which is what makes ``np.concatenate`` able to join them into a single long array -- concatenating arrays of DIFFERENT shapes directly (without raveling first) would fail.

``sum(... for W in [...])`` (a generator expression inside ``sum()``)

Computes the L1 or L2 penalty across all six weight tensors without a separate accumulator variable or explicit for-loop -- ``sum()`` consumes the generator, adding each weight tensor's own penalty contribution as it goes.

End of Day 51 syntax reference. For the concepts these lines implement, see the main Day 51 study guide PDF.